Organizacja Systemów Komputerowych - Laboratorium 1.

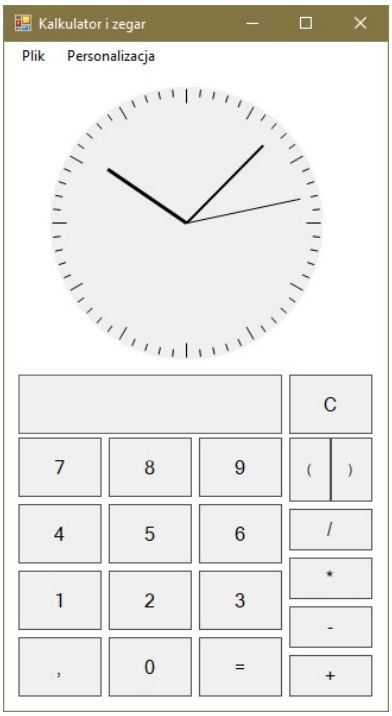
Bartosz Zwoliński 198017, Jan Wiśniewski 197582

14 kwietnia 2026

1 Opis realizowanego zadania

Opracowano kalkulator wyposażony w zegar. Aplikacja umożliwia rozwiązywanie równań wprowadzanych za pomocą przycisków dostępnych w oknie programu. Zaimplementowano dwa tryby wyświetlania zegara: analogowy oraz cyfrowy.

Program został zrealizowany w technologii Windows Forms z wykorzystaniem języka C#. Aplikacja działa wyłącznie na komputerach z systemem operacyjnym Windows.



Rysunek 1: Domyślny wygląd programu

2 Metody implementacji

2.1 Kalkulator

Obliczanie równania wprowadzonego do kalkulatora realizowane jest poprzez przekształcenie zapisu infiksowego do odwrotnej notacji polskiej, a następnie wykonanie obliczeń dla tak przygotowanego wyrażenia. Zastosowano takie rozwiązanie, aby umożliwić obsługę równań zawierających wiele operatorów.

Algorytm konwersji zapisu równania polega na umieszczaniu operatorów na stosie, skąd są one następnie zdejmowane zgodnie z kolejnością wykonywania działań oraz występowaniem liczb w równaniu. Tak przetworzone wyrażenie jest następnie obliczane, a wynik wyświetlany w oknie, w którym wprowadzono równanie.

2.2 Interfejs

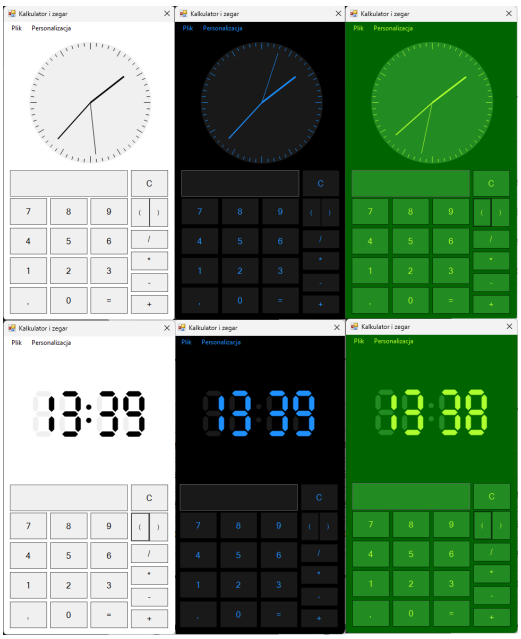
Wprowadzanie danych do kalkulatora odbywa się za pomocą przycisków umieszczonych pod zegarem. Dostępne są przyciski umożliwiające wprowadzanie cyfr, przecinka oraz operatorów dodawania, odejmowania, mnożenia, dzielenia i nawiasów. Zaimplementowano również przycisk *C*, który usuwa zawartość pola wejściowego.

```
private List<string> Convert_To_RPN(object sender, EventArgs e)
{
    Label label = sender as Label;
    Stack<char> stack = new Stack<char>();
    string expression = label.Text;
    List<string> output = new List<string>();
    StringBuilder number = new StringBuilder();

    for (int i = 0; i < expression.Length; i++)
    {
        char c = expression[i];

        if (char.IsDigit(c) || c == ',' || (c == '-'  & number.Length == 0)) number.Append(c);
        else
        {
            if (number.Length > 0)
            {
                output.Add(number.ToString());
                number.Clear();
            }

            if (c == '+' || c == '-' || c == '*' || c == '/')
            {
                while (stack.Count > 0 && stack.Peek() != '(' && GetPrecedence(stack.Peek()) >=
                    GetPrecedence(c))
                {
                    output.Add(stack.Pop().ToString());
                }
                stack.Push(c);
            }
            else if (c == '(') stack.Push(c);
            else if (c == ')')
            {
                try
                {
                    while (stack.Count > 0 && stack.Peek() != '(')
                    {
                        output.Add(stack.Pop().ToString());
                    }

                    stack.Pop();
                }
                catch
                {
                    this.valid_RPN = false;
                    return null;
                }
            }
            else if (c != ' ')
            {
                this.valid_RPN = false;
                return null;
            }
        }
    }

    if (number.Length > 0) output.Add(number.ToString());

    while (stack.Count > 0)
    {
        if (stack.Peek() == '(')
        {
            this.valid_RPN = false;
            return null;
        }
        output.Add(stack.Pop().ToString());
    }

    this.valid_RPN = true;
    return output;
}
private int GetPrecedence(char op)
{
    switch (op)
    {
        case '+':
        case '-':
            return 1;
        case '*':
        case '/':
            return 2;
        default:
            return 0;
    }
}
```

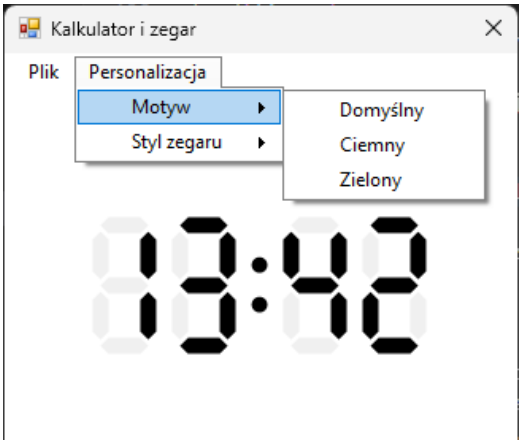
Rysunek 2: Realizacja konwersji z zapisu infiksowego do odwrotnej notacji polskiej (ONP)

```
private float Solve_RPN(List<string> rpn)
{
    float result = 0;
    Stack<float> stack = new Stack<float>();
    string number = null;
    if (rpn == null) return 0;
    for (int i = 0; i < rpn.Count; i++)
    {
        string element = rpn[i];
        if (float.TryParse(element,out _))
        {
            number = element;
        }
        if (!string.IsNullOrEmpty(number))
        {
            stack.Push(float.Parse(number.Replace(',','.')));
            number = null;
        }
        if ((element == "+" || element == "-" || element == "*" || element == "/") & stack.Count > 0)
        {
            float a = stack.Pop();
            float b = stack.Pop();
            switch (element)
            {
                case "+":
                    result = b + a;
                    break;
                case "-":
                    result = b - a;
                    break;
                case "*":
                    result = b * a;
                    break;
                case "/":
                    result = b / a;
                    break;
            }
            stack.Push(result);
        }

    }
    if (stack.Count > 0) return stack.Pop();
    else return float.Parse(number);
}
```

Rysunek 3: Realizacja rozwiązania równania w postaci ONP



Rysunek 4: Zestawienie wszystkich dostępnych motywów i zegarów



Rysunek 5: Menu wyboru motywu

Dodatkowo program obsługuje wprowadzanie danych z klawiatury. Oprócz działania odpowiadającego przyciskom widocznym na ekranie, klawisz Enter pełni funkcję znaku „=”, natomiast klawisz Backspace umożliwia usuwanie znaków pojedynczo z pola wejściowego.

2.3 Zegary

Zegar analogowy posiada trzy wskazówki wskazujące aktualną godzinę, minutę oraz sekundę, a także znaczniki ułatwiające odczyt czasu. Jest on rysowany na podstawie kąta obrotu poszczególnych wskazówek, wyznaczanego dla bieżącej chwili.

Zegar cyfrowy składa się z czterech siedmiosegmentowych wyświetlaczy prezentujących godzinę i minutę. Każdy segment może znajdować się w jednym z dwóch stanów: włączonym lub wyłączonym. Na podstawie aktualnego czasu aktywowane są odpowiednie segmenty zgodnie z zapisanymi kombinacjami. Pomiędzy wyświetlaczami znajduje się dwukropek, który pulsuje co sekundę.

Aktualny czas odczytywany jest z wykorzystaniem obiektu *Timer*.

W celu umożliwienia użytkownikowi zmiany wyglądu programu oraz typu zegara zaimplementowano menu, w którym znajdują się odpowiednie opcje. Zaimplementowano motywy: domyślny, ciemny i zielony oraz wcześniej opisane style zegarów analogowy i cyfrowy.

3 Podsumowanie i potencjalne poprawki

Program realizuje założone funkcje. Zastosowana metoda implementacji kolorystyki okna umożliwia łatwe dodawanie kolejnych stylów, co zwiększa możliwości personalizacji interfejsu przez użytkownika.

Do głównych ograniczeń programu należy brak obsługi niektórych operatorów matematycznych, takich jak pierwiastek, logarytm czy potęga. Ulepszenie aplikacji mogłoby również polegać na dodaniu kursora do pola wprowadzania danych, co pozwoliłoby na swobodniejszą edycję równania w dowolnym miejscu.